

Here is the complete, definitive, A-to-Z deployment and recovery blueprint for your encrypted multi-drive storage vault. This single-chat reference manual combines drive encryption, storage pooling, Docker container integration, startup automation, backup safety, and emergency data rescue.

Hardware Inventory & Strategy Review

- **Physical Drives:** 1× 256GB Pen Drive, 1× 120GB SSD, 1× 1TB HDD, 1× 512GB HDD.
- **Security Standard:** LUKS (Linux Unified Key Setup) AES-256 operating system level encryption.
- **Pooling Layer:** MergerFS fuses all drives into a single **1.88TB** virtual pool (/mnt/storage_pool).
- **Container Access:** Applications (Immich, Nextcloud, Filebrowser, Samba, Home Assistant) access data paths inside the decrypted pool.

 **CRITICAL PERMANENT DATA LOSS WARNING:** > Encrypting a hard drive completely wipes its contents. Since your photos are currently on these drives, **you must copy all photos off your external drives to an external backup or laptop before proceeding.** Do not run Phase 2 on a drive until its files are safely backed up.

Phase 1: Cryptographic Backup & Passphrase Keys

We enforce a strict separation of keys: an internal **Keyfile** handles automatic mounts on system startup, while a **Printed Paper Wallet Passphrase** protects your master override.

1. Generate Your Printed Master Passphrase

Run this command on your laptop or server to generate a secure, random text passphrase string:

```
Bash
openssl rand -base64 48
```

Action Required: Print or write down this exact generated string on a physical piece of paper (your Paper Wallet). Store it in a secure drawer or physical safe. **Never save this text file anywhere online or on a device connected to the internet.**

Phase 2: OS Partitioning, Encryption, and Formatting

Plug all 4 external storage drives into the USB ports of your Raspberry Pi 5.

Step 1: Install System Dependencies

Update your software repositories and install the storage management tools:

```
Bash
sudo apt update
sudo apt install cryptsetup mergerfs -y
```

Step 2: Identify Drive Identifiers

Identify the Linux kernel path assigned to each disk device using:

```
Bash
lsblk
```

Note down the device letters (e.g., /dev/sda, /dev/sdb, /dev/sdc, /dev/sdd). We use /dev/sda as our standard baseline below. Repeat this step for each drive letter.

Step 3: Wipe, Encrypt, and Setup File Systems

Execute this exact sequence for all 4 drives consecutively, substituting sda with your targeted drive letters, and updating the mapper destination index (disk1, disk2, etc.) respectively.

1. **Format with LUKS Encryption:** Create the AES-256 boundary block. Enter your master printed passphrase when prompted.

```
Bash
sudo cryptsetup luksFormat /dev/sda
```

2. **Open the Encrypted Tunnel:** Expose the secure layer to the OS kernel under a mapping label.

```
Bash
sudo cryptsetup luksOpen /dev/sda disk1
```

3. **Format with EXT4 File System:** Write a native Linux EXT4 journal framework onto the mapped crypto-container.

```
Bash
sudo mkfs.ext4 /dev/mapper/disk1
```

4. **Establish Mount Directories:** Make local directories on the OS filesystem to host the independent drives.

Bash

```
sudo mkdir -p /mnt/disk1
```

```
sudo mount /dev/mapper/disk1 /mnt/disk1
```

(Complete this loop for all remaining devices: map /dev/sdb to disk2 mounted at /mnt/disk2, /dev/sdc to disk3 mounted at /mnt/disk3, and /dev/sdd to disk4 mounted at /mnt/disk4).

Phase 3: The MergerFS Storage Pool Virtualization

With all 4 disks independently decrypted and mounted, we fuse them into a single file pool directory tree.

Bash

1. Create the master pooling directory

```
sudo mkdir -p /mnt/storage_pool
```

2. Fuse the disks using MergerFS

```
sudo mergerfs /mnt/disk1:/mnt/disk2:/mnt/disk3:/mnt/disk4 /mnt/storage_pool -o
```

```
use_ino,allow_other,minfreespace=50G,fsname=mergerfs
```

- use_ino: Keeps system file descriptors accurate for Docker environments.
- allow_other: Permits containerized application profiles to write inside the shared space.
- minfreespace=50G: Dynamically moves files away from drives getting within 50GB of capacity limits to shield them from file errors.

Bash

3. Create structured spaces for application runtimes

```
sudo mkdir -p /mnt/storage_pool/immich_data
```

```
sudo mkdir -p /mnt/storage_pool/nextcloud_data
```

```
sudo mkdir -p /mnt/storage_pool/filebrowser_data
```

```
sudo mkdir -p /mnt/storage_pool/samba_shares
```

```
# 4. Correct permissions to match the default Docker uid/gid access values
sudo chown -R 1000:1000 /mnt/storage_pool/*
```

Phase 4: Full Automation of Boot Decoding

To prevent manual command entry after power grid fluctuations or standard server system updates, we generate an automated system boot script anchored to a highly protected hardware key file.

Step 1: Provision and Secure the Local Keyfile

```
Bash
sudo mkdir -p /root/.keys
sudo openssl rand -out /root/.keys/vault.key 512
sudo chmod 400 /root/.keys/vault.key
```

Setting permissions to 400 locks this script file down so that only the root user kernel can read its contents.

Step 2: Inject the Keyfile into Your Encryption Headers

Instruct your encrypted drives to accept either your master paper wallet passphrase or this internal system keyfile:

```
Bash
sudo cryptsetup luksAddKey /dev/sda /root/.keys/vault.key
sudo cryptsetup luksAddKey /dev/sdb /root/.keys/vault.key
sudo cryptsetup luksAddKey /dev/sdc /root/.keys/vault.key
sudo cryptsetup luksAddKey /dev/sdd /root/.keys/vault.key
```

(Type your master paper wallet passphrase to authorize each addition).

Step 3: Establish the Auto-Mount Initialization Script

Create the system execution file:

```
Bash
sudo nano /usr/local/bin/mount-vault.sh
```

Paste this complete initialization sequence inside:

```
Bash
#!/bin/bash
### BEGIN INIT INFO
# Provides:      mount-vault
# Required-Start: $local_fs
# Default-Start: 2 3 4 5
# Default-Stop:  0 1 6
### END INIT INFO

# 1. Unveil the cryptographic mapping layer using the localized key
cryptsetup luksOpen /dev/sda disk1 --key-file /root/.keys/vault.key
cryptsetup luksOpen /dev/sdb disk2 --key-file /root/.keys/vault.key
cryptsetup luksOpen /dev/sdc disk3 --key-file /root/.keys/vault.key
cryptsetup luksOpen /dev/sdd disk4 --key-file /root/.keys/vault.key

# 2. Attach physical drives to system partition directories
mount /dev/mapper/disk1 /mnt/disk1
mount /dev/mapper/disk2 /mnt/disk2
mount /dev/mapper/disk3 /mnt/disk3
mount /dev/mapper/disk4 /mnt/disk4

# 3. Restructure the virtual data pool mapping layer
mergerfs /mnt/disk1:/mnt/disk2:/mnt/disk3:/mnt/disk4 /mnt/storage_pool -o
use_ino,allow_other,minfreespace=50G,fsname=mergerfs

# 4. Wake up Docker instances now that paths are ready
systemctl restart docker
```

Save and close (Ctrl + O, Enter, Ctrl + X).

```
Bash
# Make the service script executable
sudo chmod +x /usr/local/bin/mount-vault.sh

# Register the execution service into the system init runlevels
sudo update-rc.d mount-vault.sh defaults
```

Phase 5: Container Application Path Integrations

Now update your system service application paths to use the encrypted pool folders instead of your internal operating system SSD drive.

1. Immich Configuration Alignment

Navigate to your Immich installation space:

```
Bash
cd ~/homelab/immich
docker compose down
nano .env
```

Modify your environment variable string to target the pool destination path:

```
Plaintext
UPLOAD_LOCATION=/mnt/storage_pool/immich_data
```

Save the file, open your core configuration compose script (nano docker-compose.yml),

ensure your resource settings match the values below to lock down your system against SSD swap wear errors, and start the app:

YAML

services:

immich-server:

container_name: immich_server

... mapping values ...

deploy:

resources:

limits:

memory: 600M # Keeps container from choking or thrashing swap

redis:

container_name: immich_redis

deploy:

resources:

limits:

memory: 64M

database:

container_name: immich_postgres

deploy:

resources:

limits:

memory: 256M

Bash

docker compose up -d

2. Nextcloud & Filebrowser Integrations

Update the volume configuration maps within their respective compose environment scripts:

YAML

```
# Inside Nextcloud docker-compose.yml configuration
volumes:
  - /mnt/storage_pool/nextcloud_data:/var/www/html/data
```

```
# Inside Filebrowser docker-compose.yml configuration
volumes:
  - /mnt/storage_pool/filebrowser_data:/srv
```

3. Samba Networking Shares File Adjustment

Update your internal system network share service profiles:

```
Bash
sudo nano /etc/samba/smb.conf
```

Update your directory paths to target the pool:

```
Plaintext
[SecureShare]
comment = Encrypted Home Network Storage
path = /mnt/storage_pool/samba_shares
browseable = yes
read only = no
guest ok = no
```

```
Bash
sudo systemctl restart smbd
```

Phase 6: Automated Memory & Storage Optimization

To preserve system health and lock down your memory channels against unexpected drive writes, register your system memory cleanup sequence.

Step 1: Create the Automated Cleanup Script

```
Bash
nano ~/autoclean.sh
```

Paste this complete system optimization block inside:

```
Bash
#!/bin/bash
# 1. Purge dead dependencies and upgrade installations
apt autoremove -y
apt clean

# 2. Reclaim logged journal space older than 3 days
journalctl --vacuum-time=3d

# 3. Clean up orphaned Docker engine image builds
docker system prune -a -f
```

```
# 4. Clear root admin temp files
rm -rf /home/redwannabil/.cache/*
```

Save and close (Ctrl + O, Enter, Ctrl + X). Then make it executable:

```
Bash
chmod +x ~/autoclean.sh
```

Step 2: Establish the 2:30 AM Cron Automation

Open the primary root automation task tracker list:

```
Bash
sudo crontab -e
```

Add this precise operational instruction sequence line at the bottom:

```
Plaintext
30 2 * * * /home/redwannabil/autoclean.sh > /home/redwannabil/autoclean.log 2>&1
```

Phase 7: Emergency Recovery Procedures

If your system encounters a hardware issue, this section shows you how to rescue your files on any machine using your printed master passphrase.

Recovery Scenario A: Extracting Files on Another Linux Computer

1. Plug any of your encrypted drives into a recovery machine's USB port.
2. Open a terminal instance and access the encrypted content array:

Bash

```
sudo cryptsetup luksOpen /dev/sda rescue-disk
```

3. Type the master backup password string from your physical printed paper wallet.
4. Mount the partition to access the files directly:

Bash

```
sudo mkdir -p /mnt/recovery
```

```
sudo mount /dev/mapper/rescue-disk /mnt/recovery
```

Your photos will instantly appear inside /mnt/recovery, fully accessible and clean.

Recovery Scenario B: Extracting Files on a Windows Computer

1. Download and install the open-source filesystem extraction tool **LibreCrypt** or **VeraCrypt** (with LUKS compilation support enabled).
2. Plug your encrypted external drive into your Windows machine's USB slot.
3. Open the software, click **Open Container / Device**, and select your target USB device letter assignment.
4. Provide the master password string from your physical paper wallet.
5. The software tool will safely map the partition window to a traditional drive volume identifier index (like E: or F:), allowing you to interact with your photos inside Windows File Explorer.